



MAY 6, 2026

OC: SE FORMATIVE ASSESSMENT 2

DEBUGGING SOURCE CODE

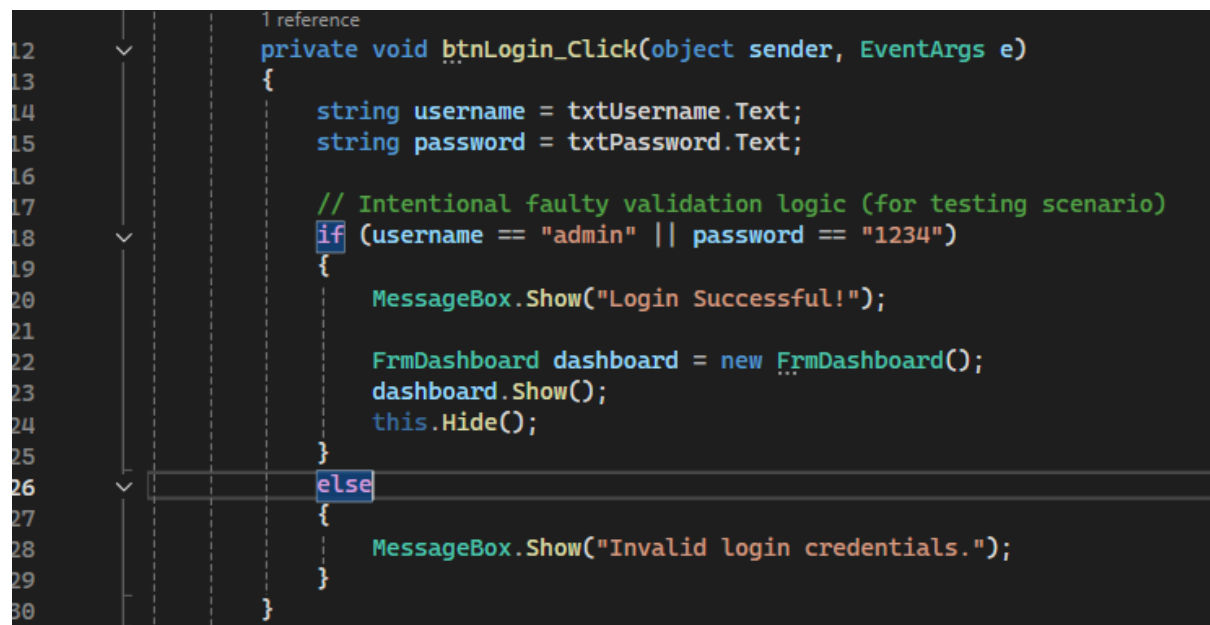
NALEDI MOFOKENG
20240778



The following defects were identified in the ULMS codebase during code review and test execution. For each defect, the original faulty code is presented, followed by the identified issue, corrected code, and explanation.

Defect 1: Incorrect Login Validation

Original Faulty Code:



```
12 | 1 reference
13 | private void btnLogin_Click(object sender, EventArgs e)
14 | {
15 |     string username = txtUsername.Text;
16 |     string password = txtPassword.Text;
17 |
18 |     // Intentional faulty validation logic (for testing scenario)
19 |     if (username == "admin" || password == "1234")
20 |     {
21 |         MessageBox.Show("Login Successful!");
22 |
23 |         FrmDashboard dashboard = new FrmDashboard();
24 |         dashboard.Show();
25 |         this.Hide();
26 |     }
27 |     else
28 |     {
29 |         MessageBox.Show("Invalid login credentials.");
30 |     }
```

Identified Issues:

1. The code uses “if (username == "admin" || password == "1234")” and the use of the OR instead of an AND is a major security flaw as the system grants a user access if the password is 1234 OR the username is admin.
2. The code uses “MessageBox.Show("Invalid login credentials.");” which does not match the system requirements. The wording misalignment creates a functional defect.
3. The code does not check whether the TextBox controls are empty and fails for empty field testing.
4. Sensitive credentials are hardcoded directly into the code rather than being stored in a database and checked using SQL queries.
5. There is a lack of exception handling.

Bug ID targeted:

- BUG#001
- BUG#002

Corrected Code and explanations:

1. The original code has a OR operator and the fix for this is using a AND operator in line 270 to provide robust authentication.

```
260     private bool AuthenticateUser(string username, string password)
261     {
262         string json = File.ReadAllText(UserFilePath);
263         var users = JsonSerializer.Deserialize<List<UserRecord>>(json);
264
265         if (users == null) return false;
266
267         foreach (var user in users)
268         {
269             if (user.Username.Equals(username, StringComparison.OrdinalIgnoreCase)
270                 && user.Pin == password)
271             {
272                 return true;
273             }
274         }
275         return false;
276     }
```

2. The message wording misalignment is fixed by updating the MessageBox to match the system requirements.

```
161     }
162     else
163     {
164         MessageBox.Show(
165             $"Incorrect username or password.\n\n" +
166             $"Please check your credentials and try again.\n\n" +
167             $"Warning: {attemptsLeft} attempt(s) remaining before your account is locked.",
168             "Login Failed",
169             MessageBoxButtons.OK,
170             MessageBoxIcon.Error);
171     }
```

3. The system processed blank fields as failed login attempts so now the string.IsNullOrEmpty checks the TextBox at the beginning of the click event and intercepts empty inputs before they reach the authentication logic.

```
50     //Empty field validation
51     if (string.IsNullOrEmpty(username) && string.IsNullOrEmpty(password))
52     {
53         MessageBox.Show(
54             "Username and password cannot be empty.\n\n" +
55             "Please enter your credentials to continue.",
56             "Missing Credentials",
57             MessageBoxButtons.OK,
58             MessageBoxIcon.Warning);
59         txtUsername.Focus();
60         return;
61     }
```

4. The hardcoded credentials are now being read from an external JSON-based user store.

```
19     //User store (simple JSON file)
20     private const string UserFilePath = "users.json";
```

```

258     private bool AuthenticateUser(string username, string password)
259     {
260         string json = File.ReadAllText(UserFilePath);
261         var users = JsonSerializer.Deserialize<List<UserRecord>>(json);
262
263         if (users == null) return false;
264
265         foreach (var user in users)
266         {
267             if (user.Username.Equals(username, StringComparison.OrdinalIgnoreCase)
268                 && user.Pin == password)
269             {
270                 return true;
271             }
272         }
273         return false;
274     }

```

5. The entire login logic is wrapped in a try-catch block with handlers for the file, JSON as well as access errors.

```

29     private void btnLogin_Click(object sender, EventArgs e)
30     {
31         try
32         {
33             //Checks lockout FIRST before anything else
34             if (IsLockedOut())
35             {
36                 return;
37             }
38
39             string username = txtUsername.Text.Trim();
40             string password = txtPassword.Text.Trim();
41
42             //Empty field validation
43             if (string.IsNullOrEmpty(username) && string.IsNullOrEmpty(password))
44             {
45                 return;
46             }
47
48             if (string.IsNullOrEmpty(username))
49             {
50                 MessageBox.Show("Username is required");
51                 txtUsername.Focus();
52                 return;
53             }
54
55             if (string.IsNullOrEmpty(password))
56             {
57                 return;
58             }
59
60             //Login logic
61             bool authenticated = AuthenticateUser(username, password);
62
63             if (authenticated)
64             {
65                 //Successful login
66                 //...
67             }
68             else
69             {
70                 //Login failed
71                 //...
72             }
73         }
74         catch (FileNotFoundException)
75         {
76             MessageBox.Show("User file not found");
77         }
78         catch (JsonException)
79         {
80             MessageBox.Show("Invalid JSON");
81         }
82         catch (UnauthorizedAccessException)
83         {
84             MessageBox.Show("Access denied");
85         }
86         catch (Exception ex)
87         {
88             MessageBox.Show("An error occurred: " + ex.Message);
89         }
90     }

```

Defect 2: Course Enrolment Duplication

Original Code:

```
19 private void btnEnroll_Click(object sender, EventArgs e)
20 {
21     // Intentional weak business-rule validation for testing purposes
22     Enrollment enrollment = new Enrollment
23     {
24         StudentId = txtEnrollStudentId.Text,
25         StudentName = txtEnrollStudentName.Text,
26         CourseName = cmbCourse.Text,
27         Semester = cmbSemester.Text
28     };
29
30     txtEnrollmentOutput.Text =
31         "Enrollment completed successfully!" + Environment.NewLine +
32         "Student ID: " + enrollment.StudentId + Environment.NewLine +
33         "Student Name: " + enrollment.StudentName + Environment.NewLine +
34         "Course: " + enrollment.CourseName + Environment.NewLine +
35         "Semester: " + enrollment.Semester;
36 }
```

Identified Issues:

1. Functional testing showed that a single student can be enrolled in the same course multiple times:

Bug ID targeted:

- BUG#003

Corrected code and explanations:

```
66 //Student must exist in registration
67 if (!StudentDataStore.StudentExists(studentId))
68 {
69     MessageBox.Show(
70         $"Student ID '{studentId}' is not registered in the system.\n\n" +
71         "Please register the student before enrolling.",
72         "Student Not Found",
73         MessageBoxButtons.OK, MessageBoxIcon.Warning);
74     txtEnrollStudentId.Focus(); return;
75 }
76
77 //Duplicate enrolment check
78 if (StudentDataStore.IsAlreadyEnrolled(studentId, courseName, semester))
79 {
80     MessageBox.Show(
81         $"Student '{studentName}' (ID: {studentId}) is already enrolled in\n" +
82         $"'{courseName}' for {semester}.\n\n" +
83         "Duplicate enrolments are not permitted.",
84         "Duplicate Enrolment",
85         MessageBoxButtons.OK, MessageBoxIcon.Warning);
86     return;
87 }
```

1. The system now first ensures that a student is registered in the database before an enrolment can proceed.
2. The system then goes through the student data store to see if the student along with their specific details already exist.

```

29 //Empty field validation
30 if (string.IsNullOrEmpty(studentId))
31 {
32     MessageBox.Show(
33         "Student ID cannot be empty.\n\nPlease enter a valid Student ID.",
34         "Missing Student ID",
35         MessageBoxButtons.OK, MessageBoxIcon.Warning);
36     txtEnrollStudentId.Focus(); return;
37 }
38
39 if (string.IsNullOrEmpty(studentName))
40 {
41     MessageBox.Show(...);
42     txtEnrollStudentName.Focus(); return;
43 }
44
45 if (string.IsNullOrEmpty(courseName))
46 {
47     MessageBox.Show(...);
48     cmbCourse.Focus(); return;
49 }
50
51 if (string.IsNullOrEmpty(semester))
52 {
53     MessageBox.Show(...);
54     cmbSemester.Focus(); return;
55 }
56
57
58
59
60
61
62
63
64

```

3. The system has implemented empty field validation and exception handling.

```

110 catch (Exception ex)
111 {
112     MessageBox.Show(
113         $"An unexpected error occurred during enrolment.\n\nDetails: {ex.Message}",
114         "Unexpected Error",
115         MessageBoxButtons.OK, MessageBoxIcon.Error);
116 }

```

Defect 3: Student Registration Duplication

Original Code: Student Registration

```

20 private void btnSaveStudent_Click(object sender, EventArgs e)
21 {
22     // Intentional weak validation for testing purposes
23     Student student = new Student
24     {
25         StudentId = txtStudentId.Text,
26         FullName = txtFullName.Text,
27         Email = txtEmail.Text,
28         Age = int.Parse(txtAge.Text),
29         Programme = cmbProgramme.Text
30     };
31
32     txtStudentOutput.Text =
33         "Student saved successfully!" + Environment.NewLine +
34         "Student ID: " + student.StudentId + Environment.NewLine +
35         "Full Name: " + student.FullName + Environment.NewLine +
36         "Email: " + student.Email + Environment.NewLine +
37         "Age: " + student.Age + Environment.NewLine +
38         "Programme: " + student.Programme;
39 }
40

```

Identified Issues:

1. Although not specifically tested, system testing revealed that there was duplication in the student registration.
2. There is no validation for empty fields.
3. Using `int.Parse()` on the Age field caused a `FormatException` and system crash if text is entered.
4. The system did not validate the format of the email address.

Corrected code and explanation:

```

108 //Duplicate student ID check
109 if (StudentDataStore.StudentExists(studentId))
110 {
111     MessageBox.Show(
112         $"Student ID '{studentId}' is already registered in the system.\n\n" +
113         "Please use a unique Student ID.",
114         "Duplicate Student ID",
115         MessageBoxButtons.OK, MessageBoxIcon.Warning);
116     txtStudentId.Focus(); return;
117 }
118
119 //Save to data store
120 Student student = new Student
121 {
122     StudentId = studentId,
123     FullName = fullName,
124     Email = email,
125     Age = age,
126     Programme = programme,
127     Status = "Active",
128     RegisteredOn = DateTime.Now
129 };
130
131 StudentDataStore.AddStudent(student);

```

1. There is a `StudentDataStore` to prevent duplicate registration attempts and the system checks the entered student ID as well to prevent duplication.

```

31 //Empty field validation
32 if (string.IsNullOrEmpty(studentId))
33 {
34     MessageBox.Show(...);
38     txtStudentId.Focus(); return;
39 }
40
41 if (string.IsNullOrEmpty(fullName))
42 {
43     MessageBox.Show(...);
47     txtFullName.Focus(); return;
48 }
49
50 if (string.IsNullOrEmpty(email))
51 {
52     MessageBox.Show(...);
56     txtEmail.Focus(); return;
57 }
58
59 if (string.IsNullOrEmpty(ageText))
60 {
61     MessageBox.Show(...);
65     txtAge.Focus(); return;
66 }
67
68 if (string.IsNullOrEmpty(programme))
69 {
70     MessageBox.Show(...);
74     cmbProgramme.Focus(); return;
75 }

```

2. The system now has validation for empty fields.

```
77 //Age must be numeric
78 if (!int.TryParse(ageText, out int age))
79 {
80     MessageBox.Show(
81         "Age must be a whole number.\n\n" +
82         "Please enter a valid numeric age (e.g. 21).",
83         "Invalid Age",
84         MessageBoxButtons.OK, MessageBoxIcon.Warning);
85     txtAge.Clear(); txtAge.Focus(); return;
86 }
87
88 if (age < 16 || age > 100)
89 {
90     MessageBox.Show(
91         "Age must be between 16 and 100.\n\n" +
92         "Please enter a valid age.",
93         "Age Out of Range",
94         MessageBoxButtons.OK, MessageBoxIcon.Warning);
95     txtAge.Clear(); txtAge.Focus(); return;
96 }
```

3. Int.Parse() is replaced with int.TryParse() and added a range to prevent system crashes.

```
98 //Email format validation
99 if (!email.Contains("@") || !email.Contains(".", "."))
100 {
101     MessageBox.Show(
102         "Please enter a valid email address.\n\nExample: student@ulms.ac.za",
103         "Invalid Email",
104         MessageBoxButtons.OK, MessageBoxIcon.Warning);
105     txtEmail.Focus(); return;
106 }
```

4. The system now has an email address validation logic.

Defect 4: Incorrect Average Calculation

Original Code:

```
31 record.Average = record.Subject1 + record.Subject2 + record.Subject3 / 3;
32
33 if (record.Average >= 50)
34 {
35     record.ResultStatus = "PASS";
36 }
37 else
38 {
39     record.ResultStatus = "FAIL";
40 }
```

Identified Issues:

1. The system produced incorrect averages.
2. The system crashed immediately when entering non-numeric input into numeric fields

Bug ID targeted:

- BUG#004

- BUG#005

Corrected code and explanation:

1. The calculation logic has been reviewed and fixed. It implements precision control using `Math.Round()`, and there is logical summation meaning the subjects in the parenthesis will be calculated first.

```
134 //Correct average calculation
135 double average = Math.Round(
136     (subject1 + subject2 + subject3) / 3.0, 2);
137
138 string resultStatus = average >= 50 ? "PASS" : "FAIL";
```

2. A `double.TryParse()` has been implemented to handle non-numeric values and prevent crashes

```
61 //Numeric validation for each subject
62 if (!double.TryParse(sub1Text, out double subject1))
63 {
64     MessageBox.Show(
65         "Subject 1 mark must be a numeric value.\n\n" +
66         "Please enter a number between 0 and 100.",
67         "Invalid Subject 1 Mark",
68         MessageBoxButtons.OK, MessageBoxIcon.Warning);
69     txtSubject1.Clear(); txtSubject1.Focus(); return;
70 }
71
72 if (!double.TryParse(sub2Text, out double subject2))
73 {
74     MessageBox.Show(...);
75     txtSubject2.Clear(); txtSubject2.Focus(); return;
76 }
77
78 if (!double.TryParse(sub3Text, out double subject3))
79 {
80     MessageBox.Show(...);
81     txtSubject3.Clear(); txtSubject3.Focus(); return;
82 }
83
84
85
86
```

3. There is range validation for when testing with the boundary value

```
92 //Range validation (0 - 100)
93 if (subject1 < 0 || subject1 > 100)
94 {
95     MessageBox.Show(
96         $"Subject 1 mark of {subject1} is out of range.\n\n" +
97         "Marks must be between 0 and 100.",
98         "Subject 1 Out of Range",
99         MessageBoxButtons.OK, MessageBoxIcon.Warning);
100     txtSubject1.Clear(); txtSubject1.Focus(); return;
101 }
102
103 if (subject2 < 0 || subject2 > 100)
104 {
105     MessageBox.Show(...);
106     txtSubject2.Clear(); txtSubject2.Focus(); return;
107 }
108
109 if (subject3 < 0 || subject3 > 100)
110 {
111     MessageBox.Show(...);
112     txtSubject3.Clear(); txtSubject3.Focus(); return;
113 }
114
115
116
117
```

4. The system has implemented empty field validation.

```

30 //Empty field validation
31 if (string.IsNullOrEmpty(studentId))
32 {
33     MessageBox.Show(
34         "Student ID cannot be empty.",
35         "Missing Student ID",
36         MessageBoxButtons.OK, MessageBoxIcon.Warning);
37     txtMarkStudentId.Focus(); return;
38 }
39
40 if (string.IsNullOrEmpty(studentName))
41 {
42     MessageBox.Show(
43         "Student Name cannot be empty.",
44         "Missing Student Name",
45         MessageBoxButtons.OK, MessageBoxIcon.Warning);
46     txtMarkStudentName.Focus(); return;
47 }
48
49 if (string.IsNullOrEmpty(sub1Text) ||
50     string.IsNullOrEmpty(sub2Text) ||
51     string.IsNullOrEmpty(sub3Text))
52 {
53     MessageBox.Show(
54         "All three subject marks must be entered.\n\n" +
55         "Please fill in Subject 1, Subject 2, and Subject 3.",
56         "Missing Marks",
57         MessageBoxButtons.OK, MessageBoxIcon.Warning);
58     return;
59 }

```

Defect 5: Reports

Original Code:

```

18 private void btnGenerateReport_Click(object sender, EventArgs e)
19 {
20     // Intentional weak validation and slow processing for testing purposes
21     string reportType = cmbReportType.Text;
22     string studentId = txtReportStudentId.Text;
23
24     // Intentional poor performance simulation
25     Thread.Sleep(4000);

```

```

27     StringBuilder report = new StringBuilder();
28
29     report.AppendLine("==== ULMS REPORT =====");
30     report.AppendLine("Report Type: " + reportType);
31     report.AppendLine("Student ID Filter: " + studentId);
32     report.AppendLine("Generated On: " + DateTime.Now);
33     report.AppendLine();
34
35     if (reportType == "Student Summary Report")
36     {
37         report.AppendLine("Student Name: John Doe");
38         report.AppendLine("Programme: Software Engineering");
39         report.AppendLine("Status: Active");
40     }
41     else if (reportType == "Marks Report")
42     {
43         report.AppendLine("Subject 1: 78");
44         report.AppendLine("Subject 2: 65");
45         report.AppendLine("Subject 3: 80");
46         report.AppendLine("Average: 169");
47     }
48     else if (reportType == "Enrollment Report")
49     {
50         report.AppendLine("Course 1: Programming 1");
51         report.AppendLine("Course 2: Database Systems");
52         report.AppendLine("Semester: Semester 1");
53     }
54     else
55     {
56         report.AppendLine("No report type selected.");
57     }

```

Identified Issues:

1. The report uses hardcoded information rather than the one a user is looking for, resulting in poor performance.

Bug ID targeted:

- BUG#006

Corrected code and explanations:

1. The system now fetches the student data from StudentDataStore based on the user's input rather than using hardcoded credentials

```
47         Student? student = StudentDataStore.GetStudentById(studentId);
48
49         if (student == null)
50         {
51             MessageBox.Show(
52                 $"No student found with ID '{studentId}'.\n\n" +
53                 "Please check the Student ID and try again.",
54                 "Student Not Found",
55                 MessageBoxButtons.OK, MessageBoxIcon.Warning);
56             txtReportStudentId.Focus(); return;
57         }
```

2. The system now prints the reports properly according to user input

```
60         StringBuilder report = new StringBuilder();
61         report.AppendLine(" ");
62         report.AppendLine("          ULMS REPORT          ");
63         report.AppendLine(" ");
64         report.AppendLine("Report Type   : " + reportType);
65         report.AppendLine("Generated On  : " + DateTime.Now.ToString("yyyy/MM/dd HH:mm:ss"));
66         report.AppendLine(" ");
```

```
68         //Student Summary Report
69         if (reportType == "Student Summary Report")
70         {
71             var enrolments = StudentDataStore.GetEnrolmentsByStudent(studentId);
72             double overall = StudentDataStore.CalculateOverallAverage(studentId);
73
74             report.AppendLine("STUDENT DETAILS");
75             report.AppendLine("-----");
76             report.AppendLine("Student ID       : " + student.StudentId);
77             report.AppendLine("Full Name        : " + student.FullName);
78             report.AppendLine("Email            : " + student.Email);
79             report.AppendLine("Age              : " + student.Age);
80             report.AppendLine("Programme        : " + student.Programme);
81             report.AppendLine("Status           : " + student.Status);
82             report.AppendLine("Registered On    : " + student.RegisteredOn.ToString("yyyy/MM/dd"));
83             report.AppendLine();
84             report.AppendLine("Total Enrolments : " + enrolments.Count);
85             report.AppendLine("Overall Average  : " + overall.ToString("F2") + "%");
86         }
```

```

88 //Marks Report
89 else if (reportType == "Marks Report")
90 {
91     var marks = StudentDataStore.GetMarksByStudent(studentId);
92
93     report.AppendLine("MARKS REPORT");
94     report.AppendLine("Student : " + student.FullName);
95     report.AppendLine("_____");
96
97     if (marks.Count == 0)
98     {
99         report.AppendLine("No marks have been captured for this student yet.");
100     }
101     else
102     {
103         foreach (var mark in marks)
104         {
105             report.AppendLine("Course      : " + mark.CourseName);
106             report.AppendLine("Semester   : " + mark.Semester);
107             report.AppendLine("Subject 1   : " + mark.Subject1);
108             report.AppendLine("Subject 2   : " + mark.Subject2);
109             report.AppendLine("Subject 3   : " + mark.Subject3);
110             report.AppendLine("Average     : " + mark.Average.ToString("F2") + "%");
111             report.AppendLine("Result      : " + mark.ResultStatus);
112             report.AppendLine("Captured On : " + mark.CapturedOn.ToString("yyyy/MM/dd"));
113             report.AppendLine("_____");
114         }
115
116         double overall = StudentDataStore.CalculateOverallAverage(studentId);
117         report.AppendLine("OVERALL AVERAGE : " + overall.ToString("F2") + "%");
118     }
119 }

```

```

121 //Enrolment Report
122 else if (reportType == "Enrollment Report")
123 {
124     var enrolments = StudentDataStore.GetEnrolmentsByStudent(studentId);
125
126     report.AppendLine("ENROLMENT REPORT");
127     report.AppendLine("Student : " + student.FullName);
128     report.AppendLine("_____");
129
130     if (enrolments.Count == 0)
131     {
132         report.AppendLine("This student is not enrolled in any courses.");
133     }
134     else
135     {
136         int count = 1;
137         foreach (var enrolment in enrolments)
138         {
139             report.AppendLine($"Enrolment {count}");
140             report.AppendLine("Course      : " + enrolment.CourseName);
141             report.AppendLine("Semester    : " + enrolment.Semester);
142             report.AppendLine("Enrolled On : " + enrolment.EnrolledOn.ToString("yyyy/MM/dd"));
143             report.AppendLine("_____");
144             count++;
145         }
146         report.AppendLine("Total Enrolments : " + enrolments.Count);
147     }
148 }
149
150 else
151 {
152     report.AppendLine("The selected report type is not recognised.");
153     report.AppendLine("Please select a valid option from the dropdown.");
154 }

```

- With regards to the performance issue, the system implemented an initialized StringBuilder and uses report.AppendLine() and report.ToString() to avoid creating multiple string objects in memory during the build process.

```

60     StringBuilder report = new StringBuilder();

```

```

146     report.AppendLine("Total Enrolments : " + enrolments.Count);

```

- The system has implemented exception handling.

```

158     catch (Exception ex)
159     {
160         MessageBox.Show(
161             $"An unexpected error occurred while generating the report.\n\nDetails: {ex.Message}",
162             "Report Error",
163             MessageBoxButtons.OK, MessageBoxIcon.Error);
164     }

```